Herald College, Kathmandu

Artificial Intelligence and Machine Learning

6CS012

Final Integrated Assessment.

Theory, Vision and Language Tasks.

April, 2026

About Document:

This document provides a comprehensive overview of the final assessment requirements, which are divided into three key components:

- **Part I – Question and Answer (QnA)** to evaluate conceptual understanding,
- **Part II – Computer Vision Task** to assess practical implementation skills,
- **Part III – Natural Language Processing Task** to examine proficiency in language-based AI applications.

You are also expected to submit an very short proposal for details on proposals see Appendix (Section 7).

Contents

1 Part - I: Question and Answer **2**
- 1.1 Assignment Details and Submission Guidelines: . . . 2
- 1.2 About Assignment: . . . 2
- 1.3 Questions: . . . 3

2 Part - II: Vision Tasks **7**
- 2.1 Assignment Details and Submission Guidelines . . . 7
- 2.2 Assignment Overview . . . 8
- 2.3 Dataset Selection: . . . 9
- 2.4 Tasks - To - Do: . . . 10
- 2.5 Part A: Designing and Analyzing Convolutional Neural Networks from Scratch. . . . 10
- 2.6 Part B: Fine-Tuning a Pre-Trained Model (Transfer Learning). . . . 12

3 (Vision Task) Reference and Guidelines for Final Report. **14**

4 Part - III: Language Tasks **17**
- 4.1 Assignment Details and Submission Guidelines . . . 17
- 4.2 Assignment Overview . . . 18
- 4.3 About Assignment: . . . 18
- 4.4 Dataset Selection: . . . 19
- 4.5 Tasks - To - Do: . . . 20

5 (Language Task) References and Guidelines for Final Report **22**

6 Appendix **25**
- 6.1 Sample Implementation for the use of word2vec with gensim: . . . 25

7 Project Proposal **27**
- 7.1 Proposal Requirements . . . 27
- 7.2 Guidelines and References for the Proposal . . . 27

8 Grading and Marking Breakdown **30**
- 8.1 Overall Assessment Breakdown: . . . 30
- 8.2 Recommended Performance Band: . . . 30

9 Making a Final Submission: **31**

Part - I - QnA.

1 Part - I: Question and Answer

1.1 Assignment Details and Submission Guidelines:

1.1.1 Assignment Details:

Due	Marks	Submission
To be Announced	10	A Report.

1.1.2 Plagiarism and AI Generated Content

Plagiarism of more than 20% and any AI-generated content found in the report will be reported for academic misconduct. Thus, we highly encourage you to submit your original work.

1.1.3 Submission Guidelines:

You are required to submit a well-organized and comprehensive report that thoroughly addresses all questions outlined in each component of the assessment.

1.1.4 Naming Convention:

`File Name: WLVID_FullName(firstname+last)_QnA.pdf`

1.2 About Assignment:

This assessment consists of two sections: Long Question and Answer and Short Question and Answer. In the Long Question section, students are required to answer any one question. The Short Question section comprises two subsections, each containing two questions; students must answer one question from each subsection.

1.3 Questions:

Long Question:: Answer any 1 [5 Marks]

1. You are a Data Scientist at a leading digital payment platform such as eSewa, working on large-scale user transaction data.
   - Identify two high-impact business problems where unsupervised learning can provide value (e.g., customer segmentation, fraud detection, behavioral profiling).
   - For each problem:
     - Clearly define the problem in a real-world context.
     - Propose a suitable unsupervised learning approach (e.g., clustering, anomaly detection, dimensionality reduction) and recommend specific algorithms (e.g., K-Means, DBSCAN, Autoencoders).
     - Discuss how the model outputs can be integrated into real-time or batch systems to support business decisions.
     - Briefly mention one limitation or challenge of your proposed approach.
   - Assume limited labeled data and large-scale transactions.
2. You are a Machine Learning Engineer at a rapidly growing e-commerce company responsible for deploying and maintaining ML systems in production.
   - Identify and explain at least three real-world challenges encountered in ML systems (e.g., data drift, model degradation, latency, scalability, monitoring).
   - For each challenge:
     - Discuss potential consequences if not properly addressed.
     - Propose practical solutions (e.g., retraining pipelines, monitoring systems, MLOps practices, distributed serving).
     - Highlight any trade-offs involved in your solution.
   - Reflect on how cross-functional collaboration (data scientists, engineers, product managers) contributes to building reliable ML systems.
   - Briefly comment on how modern tools (e.g., automation, pipelines, or LLM-based systems) may help address these challenges.

[**Expectation: For the question presented above, write a brief answer limited to 300 - 400 words.**]

Short Question:: Pick 1 from each Category [5 Marks]

1. Overfitting:: Answer any 1 [2.5 Marks]

1. In the context of machine learning:
   - Define overfitting and underfitting, and clearly distinguish between the two in terms of model behavior on training and unseen data.
   - Explain why both overfitting and underfitting negatively impact generalization performance.
   - Illustrate your explanation using simple examples (e.g., a model memorizing training data vs failing to capture complex patterns in the data).
2. Overfitting is a common challenge in deep learning, especially when models have high capacity.
   - Describe at least two techniques used to reduce overfitting (e.g., dropout, early stopping, data augmentation, L2 regularization).
   - For each technique:
     - Explain the intuition behind how it works.
     - Describe how it affects the training process or model behavior.
     - Provide a real-world application example (e.g., image classification, text classification, fraud detection).

[**Expectation: For the question presented above, write a brief answer limited to 200 words.**]

2. Neural Network Architecture:: Answer any 1 [2.5 Marks]

1. Explain the fundamental differences between a Convolutional Neural Network (CNN) and a Recurrent Neural Network (RNN):
   - In terms of:
     - Input structure and data type
     - Information flow and architecture design
     - Parameter sharing mechanism
   - Provide suitable real-world examples where each architecture is preferred (e.g., image classification for CNNs, sequential/time-series modeling for RNNs).
   - Briefly discuss common challenges encountered during training deep learning models, such as:
     - Vanishing or exploding gradients
     - Overfitting
   - For each challenge, mention at least one mitigation technique (e.g., batch normalization, gradient clipping, early stopping).

2. FCN vs. Autoencoder:
   - Compare a standard feedforward neural network (fully connected network) with an autoencoder in terms of:
     - Architecture structure
     - Objective/functionality
     - Input-output relationship
   - Provide a simple example illustrating how an autoencoder differs from a standard neural network.
   - Describe at least one application of autoencoders (e.g., dimensionality reduction, anomaly detection, image denoising).
   - Explain how autoencoders help solve the chosen problem.

[**Expectation: For the question presented above, write a brief answer limited to 200 words.**]

Part - II - Vision Tasks.

2 Part - II: Vision Tasks

2.1 Assignment Details and Submission Guidelines

2.1.1 Assignment Details:

Due	Marks	Submission
Week - 12 - Tutorial	40	A Report and Code Notebook.

2.1.2 Plagiarism and AI Generated Content

Plagiarism of more than 20% and any AI-generated content found in the report will be reported for academic misconduct. Thus, we highly encourage you to submit your original work.

2.1.3 Submission Guidelines:

- Working In Group:
  - You are allowed to work on common dataset in group of 3 members (for more than that, please take a approval from your instructor), but remember members will be assessed individually.
  - Make a presentation and present in group after the final submission.
- Individual:
  - All the members must write individual report, code independently and be able to answer the question and explain the code.
- What to Submit?
  - You are expected to submit a report based on the task, and associated code file.
  - For Code:
    1. All solutions - Code must be written in the Jupyter notebook.
    2. All codes must be pushed to `GitHub` before the deadlines.
  - For report:
    1. Please follow the APA format; for a sample, see the attached documents.
  - Where to submit?
    A Portal on Canvas.
- After Submission:
  - You are expected defend your work after the submission.Please consult with your respected instructor for date and time of the viva.

The Final Date for submission is: `To Be Announced Shortly.`

2.1.4 Naming Conventions:

You are supposed to strictly follow the naming conventions, and any file that does not follow the naming conventions will be marked as "0".
`File Name: WLVID_FullName(firstname+last).ipynb`,
`File Name: WLVID_FullName(firstname+last).pdf`

2.2 Assignment Overview

Image Classification with Convolutional Neural Network.

2.2.1 About Assignment:

In this assignment, you will undertake a comprehensive end-to-end deep learning project for image classification. The objective is to deepen your understanding of the entire deep learning pipeline, from data preprocessing to model building and evaluation. This assignment integrates knowledge from Weeks 1 to 6, challenging you to apply these concepts to real-world image classification tasks. This assignment has two parts:

- In part A: You will train and experiment with baseline CNN model from scratch.
- In part B: Fine tuned a pre - trained model just as you do in real world application.

2.2.2 Cautions!!!

In this assignment, you will perform a series of task (explained below) for Image Classification with a appropriate dataset and provide a rigorous rationale for your solutions. We will determine scores by judging both the soundness and cleanliness of your **code**, the quality of the **write-up(report)** and your ability to answer the question during **viva**. Here are examples of aspects that may lead to **point deductions:**

- Use of misleading, unnecessary, or unmotivated graphic elements.
- Unreadable code.
- Missing or incomplete design rationale in write-up.
- Ineffective encoding for your stated goal (e.g., distracting colors, improper data transformation).

Tools and Python Package which can be used for this assignments (listed but not limited to):

1. **Numpy library(np)**
2. **Matplotlib library(plt)**
3. **sickit Learn(sklearn)**
4. **Deep Learning Framework - {Recommended - Keras}.**

2.2.3 Learning Outcomes:

Learning outcomes can be following but not limited to:

1. Use numpy, keras as the primary tool to process image data in Python,
2. Use matplotlib library to produce various plots for visualization,
3. Perform some basic image manipulation
4. Understand the architecture and various layer of Convolutional Neural Network.
5. Be able to design, build and train CNN Model, with modern deep learning library (Keras) for image classification task.

2.3 Dataset Selection:

1. Please feel free to pick any datasets in image format that matches the task requirements. But please take pre-approval from your respected instructor and Module leader.
2. If you are not sure about which dataset to pick, select one from the options provided.
3. Note: No two groups in one Section can have same datasets.

The best source to find datasets are but not limited to:

1. Kaggle Datasets:
   Kaggle provides a high-quality dataset in various formats that we can easily find and download.
   https://www.kaggle.com/datasets
2. UCI Machine Learning Repository:
   This repository contains databases, domain theories, and data generators that are widely used by the machine learning community for the analysis of ML algorithms.
   https://archive.ics.uci.edu/ml/index.php.
3. Google's Dataset Search Engine:
   Google dataset search engine is a search engine launched by Google on September 5, 2018. This source helps researchers to get online datasets that are freely available for use.
   https://toolbox.google.com/datasetsearch.

2.4 Tasks - To - Do:

Please Complete all the Tasks as instructed.

2.5 Part A: Designing and Analyzing Convolutional Neural Networks from Scratch.

This section focuses on designing, training, and evaluating convolutional neural networks (CNNs) from scratch. You will first build a baseline model with a simple architecture, then extend it into a deeper model with regularization to analyze performance improvements. A thorough experimentation section will guide comparative analysis of different architectures and optimizers.

2.5.1 Data Understanding, Analysis, Visualization and Cleaning:

Before building models, analyze and understand the dataset by answering the following questions:

- What does the dataset represent? Provide a brief description.
- How many total images are in the dataset?
- What is the distribution of images across different classes?
- How is the dataset split into training and validation sets? Justify your choice.
- What preprocessing techniques (e.g., resizing, normalization) were applied?
- What data generators, if any, were used for preprocessing and augmentation?
- If data augmentation was applied, provide visualizations of sample augmented images.

2.5.2 Design, Train, and Evaluate a Baseline Model:

1. **Model Architecture::**
   - Build a baseline CNN model with the following structure:
     - Three Convolutional layers, each followed by pooling layers.
     - Three fully connected layers (FCN)
     - An output layer suitable for classification.
   - – Kernel Sizes
     - Number of Filters
     - Activation Functions
   - Provide and Interpret the model summary.
2. **Model Training:**
   - Train the baseline model for an appropriate number of epochs.
   - Plot the training vs validation loss curves over epochs.
   - Note: We recommend using Google Colab with GPU or TPU acceleration for training.

3. **Model Evaluation:**
   - Evaluate the model using appropriate metrics (e.g. accuracy, precision, recall and F-1 score).
   - Perform inference on sample images and plot the result.
   - Discuss key observations about model performance.

2.5.3 Deeper Architecture with Regularization Layer:

1. **Model Architecture:**
   - Extend the baseline model by modifying the number of filters and layers to build deeper architecture{at least double the layer in comparison to baseline model}.
   - Extend the layers introducing the following regularization techniques.
   - Print and analyze the new model summary.
2. **Model Training:**
   - Train the deeper model for an appropriate number of epochs.
   - Compare its training time and loss curves against the baseline model.
3. **Model Evaluation:**
   - Evaluate the deeper model on the same performance metrics as the baseline model.
   - Discuss improvements (or lack thereof) compared to the baseline model.

2.5.4 Experimentation and Comparative Analysis:

1. **Baseline vs. Deeper Model Performance:**
   - Compare classification accuracy, loss, and evaluation metrics.
   - Discuss whether adding more layers and filters improved performance.
2. **Computational Efficiency:**
   - Compare training time and computational cost of the two models.
   - Discuss the trade - offs between model complexity and efficiency.
3. **Optimizer Analysis: SGD vs Adam:**
   - Train the deeper model with SGD and then Adam.
   - Compare the impact of these optimizers on convergence speed and final performance.
4. **Ablation Study:**
   - Remove one component (e.g., dropout or batch normalization).
   - Analyze how performance changes.
5. **Challenges and Observations:**
   - Discuss any difficulties faced, such as overfitting and underfitting.

- Report the total training time for deeper architecture.
- Mention if hardware acceleration (e.g. Google Colab, with GPU/TPU was used.

2.6 Part B: Fine-Tuning a Pre-Trained Model (Transfer Learning).

In many real-world deep learning applications, we leverage pre-trained model weights from models trained on large-scale datasets such as ImageNet. These pre-trained models (e.g., Inception, ResNet, VGG, MobileNet) are highly effective because they have already learned rich feature representations. Transfer learning allows us to adapt these models for our specific image classification task, reducing the need for extensive labeled data and computational resources.

2.6.1 Loading and Adapting a Pre - Trained Model:

- Select an appropriate pre - trained CNN model (e.g. VGG, ResNet, Incpetion) based on the characterstics of your dataset.
- Load the pre - trained model using keras.
- Modify the model architecture to fit your specific classification task.
  - Remove the original fully connected layers.
  - Add a new dense layers tailored to your dataset's number of classes.
- Choose whether to freeze certain layers or fine - tune the entire model based on your dataset size and computational constraints.

2.6.2 Model Training and Fine - Tuning:

1. Training Strategies:
   - Feature Extraction:
     - Freeze the convolutional base of the pre - trained model to retain previously learned feature representations.
     - Train only the newly added layers for classification.
   - Fine - Tuning:
     - Unfreeze some or all convolutional layers and allow them to adjust to the new dataset.
     - This approach requires a lower learning rate to avoid overfitting and catastrophic forgetting.
2. Handling Input Size Differences:
   - ImageNet models expect a specific input image size (e.g.224×224 pixels for VGG, ResNet).
   - Resize dataset images to match the required dimensions using appropriate Data Generator in Keras.
3. Handling Output Layer Differences:

- ImageNet trained models have a 1000 - class softmax output layer.
- Modify the final classification layer to match the number of classes in your dataset.

2.6.3 Model Evaluation and Prediction:

- Evaluate the fine tuned model using the same performance metrics as in Part A.
- Make predictions on test data and compare results with the baseline and deeper models from Part A.
- Discuss whether transfer learning outperforms training from scratch for your dataset.

3 (Vision Task) Reference and Guidelines for Final Report.

This section serves as a reference for structuring your project report.

1. Title

Provide a clear and descriptive title that represents your work effectively. The title should reflect the problem being solved or the key focus of your project.

2. Abstract

The abstract should be a concise summary (150–250 words) of your work, including:

- **Objective**: What problem are you solving?
- **Methods**: Briefly describe the models/techniques used.
- **Key Findings**: Highlight significant results.
- **Conclusion & Impact**: Summarize insights and recommendations.

3. Introduction

Clearly define the problem statement and its real-world significance. Provide background information on deep learning for image classification and discuss relevant prior work. Outline the goals and scope of the project.

4. Dataset

Describe the dataset(s) used:

- Source (e.g., CIFAR-10, custom dataset).
- Number of images, classes, and resolution.
- Any preprocessing applied (e.g., resizing, normalization).

If using a custom dataset, explain:

- Collection method (e.g., web scraping, manual labeling).
- Labeling approach (manual or automated).
- Challenges faced during data collection.

5. Methodology

Detail the deep learning models used, including architecture design:

- **Baseline Model**: 3 convolutional + pooling layers, 3 FCN layers, output layer.
- **Deeper Model**: Additional layers with Batch Normalization and Dropout.

- **Loss Function**: Cross-entropy, etc.
- **Optimizer**: SGD, Adam, or other choices.
- **Hyperparameters**: Learning rate, batch size, epochs.

6. Experiments and Results

Describe the experiments performed to compare models.

6.1 Baseline vs. Deeper Architecture

Compare accuracy, loss, and training time. Analyze whether adding layers/filters improved performance.

6.2 Computational Efficiency

Compare training time and memory requirements. Discuss hardware acceleration (e.g., GPU/TPU usage in Google Colab).

6.3 Training with Different Optimizers

Train the deeper model using both SGD and Adam. Compare convergence speed and final accuracy.

6.4 Challenges in Training

Discuss issues such as overfitting, underfitting, or unstable training. Report total training time and computational cost.

7. Fine-Tuning or Transfer Learning

Describe the pre-trained model chosen (e.g., VGG, ResNet, Inception). Explain whether you used feature extraction (freezing layers) or fine-tuning. Show and discuss performance improvements using transfer learning. Provide a comparison with models trained from scratch.

8. Conclusion and Future Work

Summarize key findings and insights from the project. Discuss trade-offs between different architectures and methods. Identify limitations or failure cases in your models. Suggest potential improvements (e.g., additional datasets, hyperparameter tuning). Mention future research directions if applicable.

Part - III - Language Tasks.

4 Part - III: Language Tasks

4.1 Assignment Details and Submission Guidelines

4.1.1 Assignment Details:

Due	Marks	Submission
Week - 12 - Tutorial	25	A Report and Code Notebook.

4.1.2 Plagiarism and AI Generated Content

Plagiarism of more than 20% and any AI-generated content found in the report will be reported for academic misconduct. Thus, we highly encourage you to submit your original work.

4.1.3 Submission Guidelines:

- Working In Group:
  - You are allowed to work on common dataset in group of 3 members (for more than that, please take a approval from your instructor), but remember members will be assessed individually.
  - Make a presentation and present in group after the final submission.
  - You are expected to work with same group you formed during Assignment - 1.
- Individual:
  - All the members must write individual report, code independently and be able to answer the question and explain the code.
- What to Submit?
  - You are expected to submit a report based on the task, and associated code file.
  - For Code:
    1. All solutions - Code must be written in the Jupyter notebook.
    2. All codes must be pushed to `GitHub` before the deadlines.
  - For report:
    1. Please follow the APA format; or as per the provided sample, for sample see the attached documents.
  - Where to submit?
    A Portal on Canvas.
- After Submission:
  - You are expected defend your work after the submission.Please consult with your respected instructor for date and time of the viva.
  - It will be on Tutorial or Workshop session of Week - 12.

The Final Date for submission is: `To Be Announced Shortly.`

4.1.4 Naming Conventions:

You are supposed to strictly follow the naming conventions, and any file that does not follow the naming conventions will be marked as "0".
`File Name: WLVID_FullName(firstname+last).ipynb`,
`File Name: WLVID_FullName(firstname+last).pdf`

4.2 Assignment Overview

(Sentiment Analysis) Text Classification with RNN.

4.3 About Assignment:

In this assignment, you will undertake a comprehensive end-to-end deep learning project for text classification. The objective is to deepen your understanding of the entire NLP pipeline, from data preprocessing to model building and evaluation. This assignment integrates knowledge from text processing, tokenization, and recurrent neural networks (RNNs) to long short-term memory (LSTM) models, challenging you to apply these concepts to real-world sentiment analysis tasks.

4.3.1 Cautions!!!

In this assignment, you will perform a series of task (explained below) for Text Classification with an appropriate dataset and provide a rigorous rationale for your solutions. We will determine scores by judging both the soundness and cleanliness of your **code**, the quality of the **write-up(report)** and your ability to answer the question during **viva**. Here are examples of aspects that may lead to **point deductions:**

- Use of misleading, unnecessary, or unmotivated graphic elements.
- Unreadable code.
- Missing or incomplete design rationale in write-up.
- Ineffective encoding for your stated goal (e.g., distracting colors, improper data transformation).

Tools and Python Package which can be used for this assignments (listed but not limited to):

1. **Numpy library(np)**
2. **Matplotlib library(plt)**
3. **sickit Learn(sklearn)**
4. **Deep Learning Framework** - {**Recommended - Keras**}**.**

4.3.2 Learning Outcomes:

Learning outcomes can be following but not limited to:

1. Gain hands - on experience in building and evaluating RNN and LSTM models for text classification.
2. Learn data pre - processing techniques, including tokenization, padding, and text vectorization using embeddings.
3. Understand how to train, optimize and evaluate models with various metrics (accuracy, confusion matrix, classification report.)
4. Explore the use of pre - trained word2vec embeddings for improving LSTM model performance.
5. Develop skills in visualizing model performance, comparing different models, and identifying overfitting or underfitting.
6. Implement a real - time text class prediction interface using a basic GUI framework like Tkinter, Gradio, or Streamlit etc.

4.4 Dataset Selection:

Each group will be assigned a **specific dataset by the instructor**. These datasets will vary between groups, and each group is required to work exclusively with the dataset assigned to them for the entirety of the task. Selection of an alternative dataset is not permitted. This approach ensures that all students engage with their designated data, thereby facilitating consistent evaluation and meaningful comparison of results across groups.

4.5 Tasks - To - Do:

Please Complete all the Tasks as instructed:

4.5.1 Text Preprocessing, Tokenization, and Sequence Padding:

In this section, you will perform data preprocessing, tokenization, and padding on the raw tweet data:

- **Load the dataset**: Use `Pandas` to load the raw tweet data.
- **Clean the text**: Perform the following preprocessing steps on the text:
  - Lowercase all text to ensure uniformity.
  - Remove URLs, mentions (`@user`), hashtags (#), numbers, and special characters.
  - Handle contractions (e.g., `"don't"` → `"do not"`).
  - Remove stopwords and lemmatize words to reduce words to their base form.
- **Visualize the cleaned data**: Use tools like a word cloud or display the most frequent words to explore the cleaned data.
- **Tokenization and Padding**: Convert the cleaned text data into sequences and ensure uniform sequence lengths by applying padding:
  - Split the dataset into **80% training** and **20% testing** using `train_test_split`.
  - Use **Keras Tokenizer** to convert the text into sequences of integers.
  - Apply **padding** to ensure all sequences have the same length. Utilize percentile-based padding to avoid excessively long sequences.

4.5.2 Model Building and Training:

In this section, you will build three different models for text classification:

1. **Simple RNN** with a trainable Embedding layer.
2. **LSTM** with a trainable Embedding layer.
3. **LSTM** with pretrained `Word2Vec` embeddings.

Model Requirements:

Each module most include:

- **Embedding Layer:**
  - For model1 and 2 use Embedding layer:
    - `rnn_model.add(Embedding(input_dim= , output_dim= , input_length= ))`
  - For model 3 Use `Word2Vec` pretrained embeddings {A sample implementation is provided at the Appendix Section of this document.}

- **Recurrent Layer:** `SimpleRNN` for model 1 and `LSTM` for models 2 and 3.
- **Dense Output Layer:**

4.5.3 Model Training and Evaluation:

Now, compile, train, and evaluate your models:

- Compile models with following configurations:
  - **Appropriate Loss Function.**
  - **Appropriate optimizer.**
  - **Appropriate Metrics.**
- Train the Models for desired number of epochs using callbacks and early stopping.
- Visualization:
  - Plot the Training vs. Validation loss and accuracy over epochs.
  - Compare the performance of the Model 1 vs. Model 2 vs. Model 3.
- Evaluate the models using:
  - **Accuracy.**
  - **Confusion Matrix.**
  - **Classification Report.**

4.5.4 Error Analysis:

- Analyze misclassified examples:
  - Provide 2 - 3 incorrect predictions.
  - Evaluate Model Complexity vs. Performance.
  - Explain Possible reasons for error.
- Suggest potential improvements.

4.5.5 GUI for Real Time Prediction:

For an extra challenge, build a simple GUI using one of the following libraries:

- **Tkinter, Gradio, or Streamlit**.

Allow the user to input a tweet and get a prediction in real-time.

5 (Language Task) References and Guidelines for Final Report

This section provides guidelines for structuring your project report on text classification using Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Word2Vec embeddings.

1. Title

Provide a clear and descriptive title for your project. The title should reflect the primary objective, which is text classification using RNN, LSTM, and Word2Vec embeddings. Example: *"Sentiment Analysis of Tweets using RNN, LSTM, and Word2Vec Embeddings"*.

2. Abstract

The abstract should summarize the entire project in 150–250 words, covering:

- **Objective**: What is the task, and why is it important (e.g., sentiment analysis of tweets)?
- **Methods**: Briefly mention the models used (RNN, LSTM) and embedding techniques (Word2Vec).
- **Key Findings**: Highlight the most significant results (e.g., model accuracy, performance comparison).
- **Conclusion & Impact**: Summarize your conclusions and any insights for future work or improvements.

3. Introduction

Define the problem statement clearly (e.g., sentiment analysis of tweets for classification into positive, negative, and neutral classes). Discuss the real-world significance of text classification and the relevance of deep learning models such as RNN and LSTM for sequence data. Provide an overview of previous work in text classification and sentiment analysis.

4. Dataset

Describe the dataset(s) used:

- **Source**: Mention the dataset used (e.g., Twitter dataset, Sentiment140, or any custom dataset).
- **Data Size**: Number of tweets, classes, and the number of features (e.g., tokenized words).
- **Preprocessing**: List any preprocessing steps applied, such as text cleaning (removal of URLs, hashtags, mentions) and lemmatization.

5. Methodology

Detail the deep learning models used for text classification:

- **Text Preprocessing**: Describe the steps taken to clean and prepare the text (tokenization, stopword removal, lemmatization).

- **Model Architecture**: Outline the architectures used:
  - **Simple RNN**: Use of Recurrent Neural Networks for sequential data.
  - **LSTM**: Use of Long Short-Term Memory networks to overcome the vanishing gradient problem in RNNs.
  - **Word2Vec Embedding**: Incorporating pretrained Word2Vec embeddings for better word representation.
- **Loss Function**: Explain the loss function used (e.g., binary cross-entropy for binary classification, categorical cross-entropy for multiclass).
- **Optimizer**: Specify the optimizer used (e.g., Adam, SGD).
- **Hyperparameters**: Discuss key hyperparameters (learning rate, batch size, number of epochs).

6. Experiments and Results

Describe the experiments performed to evaluate the models.

6.1 RNN vs. LSTM Performance

Compare the performance of RNN and LSTM models in terms of accuracy, loss, and training time. Discuss the impact of using LSTM over RNN.

6.2 Computational Efficiency

Analyze the computational efficiency of training the models. Compare training time, memory usage, and hardware requirements (e.g., GPU usage on Colab).

6.3 Training with Different Embeddings

Evaluate the performance of the models when using different embeddings:

- **Random Embeddings**: Trained word embeddings.
- **Word2Vec Embeddings**: Pretrained Word2Vec embeddings.

Discuss the improvements observed with Word2Vec over random embeddings.

6.4 Model Evaluation

Discuss the evaluation metrics used:

- **Accuracy**: Overall accuracy of the model.
- **Confusion Matrix**: Use confusion matrix for detailed performance analysis (True Positives, False Positives, etc.).
- **Precision, Recall, F1-Score**: Discuss precision, recall, and F1 score for classification tasks.

7. Conclusion and Future Work

Summarize the key findings of the project and provide a discussion of the results. Highlight the performance of RNN vs. LSTM and the impact of Word2Vec embeddings. Mention any limitations observed, such as overfitting or model convergence issues. Suggest potential improvements (e.g., hyperparameter optimization, larger datasets). Discuss future work, including extensions or applications of the model.

6 Appendix

6.1 Sample Implementation for the use of word2vec with gensim:

Important Note on Using Gensim for Word2Vec Embeddings

The `gensim` library, which is required for downloading and utilizing Word2Vec embeddings, is not pre-installed in Google Colab by default. Therefore, it must be installed manually before use.

Furthermore, it is important to note that the latest versions of `numpy` may not be fully compatible with the current version of `gensim`. If compatibility issues arise (e.g., import errors or installation failures), it may be necessary to downgrade `numpy` to an earlier version.

Before using `gensim`, please execute the following commands in your Colab environment to ensure proper installation and compatibility:

Before downloading pre - trained word2vec

```
!pip install numpy==1.23.5
!pip install gensim
!pip install jax==0.4.13
```

After running these commands, restart the runtime if prompted to do so. This ensures that `gensim` and `numpy` operate correctly together within the Google Colab environment.

Word2Vec Embedding for LSTM Model

To incorporate `Word2Vec` embeddings into the LSTM model, the following steps should be followed:

Step 1: Load Word2Vec Pretrained Model

The `Word2Vec` model can be trained on your data or a pretrained version (such as Google's Word2Vec) can be used. To use Word2Vec embeddings in this project, you first need to download a pre-trained Word2Vec model. One commonly used model is the **Google News Word2Vec** model. The Google News pre-trained Word2Vec model can used directly from gensim api.load() method as shown below:

Using a pre - trained word2vec model:

```
import gensim.downloader as api
# Load Pre-trained GloVe Embeddings
embedding\_model = api.load('glove-wiki-gigaword-50') # 50-dimensional word2vec
```

Apart from the Google News embeddings, there are several other pre-trained Word2Vec models that could be used depending on the task and language requirements:

- `fasttext-wiki-news-subwords-300`
- `conceptnet-numberbatch-17-06-300`
- `word2vec-ruscorpora-300`

- `word2vec-google-news-300`
- `glove-wiki-gigaword-50`
- `glove-wiki-gigaword-100`
- `glove-wiki-gigaword-200`
- `glove-wiki-gigaword-300`
- `glove-twitter-25`
- `glove-twitter-50`
- `glove-twitter-100`
- `glove-twitter-200`

You can use above "`word2vec`" to build a embedding layer as follow:

Using word2vec for Embedding.

```
embedding_dim = 50 # Match with downloaded GloVe model dimension
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if word in embedding_model:
        embedding_vector = embedding_model[word]
        embedding_matrix[i] = embedding_vector
    else:
        # Words not found in embedding index will be all-zeros
        pass

# Build the LSTM Model with Pre-trained Embedding Layer
model = Sequential()
model.add(Embedding(
    input_dim=vocab_size,
    output_dim=embedding_dim,
    weights=[embedding_matrix],
    input_length=padded_sequences.shape[1],
    trainable=False # Set to True if you want to fine-tune embeddings
))
model.add(LSTM(64))
model.add(Dense(1, activation='sigmoid')) # or softmax if multiclass
```

7 Project Proposal

Each group must submit a project proposal before proceeding with the implementation. The proposal should focus on the dataset being used and will be presented during the Week 7 workshop.

7.1 Proposal Requirements

- Provide a detailed explanation of the dataset you have chosen, including its source.
- Present key statistics about the dataset, such as the total number of images or text instances, number of classes, and the distribution of images per class.
- Perform basic visualizations to enhance the understanding of the dataset.
- Ensure that no pre-loaded image or text datasets from deep learning frameworks (such as TensorFlow or PyTorch) are used.
- The proposal should not exceed three pages. Refer to the next section for detailed guidelines.
- Each group must also prepare a presentation summarizing their findings (not exceeding seven slides) and present it during the Week 7 workshop.

7.2 Guidelines and References for the Proposal

1. Formatting Guidelines

- Length: One to two pages (approximately 300-400 words).
- Font: Times New Roman, 12 pt.
- Margins: 1 inch on all sides.
- Line Spacing: Single.

2. Evaluation Criteria

The proposal will be evaluated based on:

- Clarity and feasibility of the research question.
- Relevance and appropriateness of the dataset.
- Adherence to formatting and submission guidelines.

3. Submission Instructions

- Submit your proposal as a PDF file (File Name Format: `GroupName_UniID.pdf`).
- Submit the proposal to your respective instructor and verify it before the Week 7 workshop.
- One proposal and presentation per group is sufficient, but all group members must be clearly mentioned.

4. Suggested Structure for Your Proposal

You can structure your proposal based on the following guidelines:

1. Research Question

Clearly define the research question you aim to answer using image classification. Examples include identifying objects in images, classifying medical scans, or recognizing handwritten characters.

2. Title

Provide a suitable title based on your research question.

3. Dataset Description

- Describe the dataset(s) used:
  - Source
  - Number of images or text instances, number of classes
- If using a custom dataset, explain:
  - Collection method (e.g., web scraping, manual labeling)
  - Labeling approach (manual or automated)
  - Challenges faced during data collection

4. Preliminary Data Analysis

Provide basic insights into the dataset:

- Class distribution (balanced or imbalanced)
- Sample visualizations:
  - Images or text examples.
- Any initial observations or challenges

Part - IV - Grading and Marking Breakdown.

8 Grading and Marking Breakdown

8.1 Overall Assessment Breakdown:

Component	**Marks**
I: QnA	10
II: Vision Task	40
Part A (From Scratch CNN)	30
Part B (Transfer Learning)	10
III: NLP Task	25
IV: Mid Term Quiz	20
V: Weekly workshops and Class Performance	5
Total	**100 Marks**

Table 1: Overall Assessment Breakdown

8.2 Recommended Performance Band:

Use percentage of allocated marks per criterion:

Level	**Score Range**	**Interpretation**
Excellent	90% – 100%	Outstanding work; complete; correct; and insightful with strong justification.
Good	75% – 89%	Mostly correct, minor gaps
Fair	65% – 74%	Reasonable understanding; Some gaps and inconsistencies
Satisfactory	50% – 64%	Basic understanding; Significant gaps but meets minimum expectations
Pass	40% – 49%	Minimal acceptable performance; limited understanding
Poor(Fail)	0% – 39%	Incomplete; incorrect; or lacks fundamental understanding

Table 2: Performance Levels

9 Making a Final Submission:

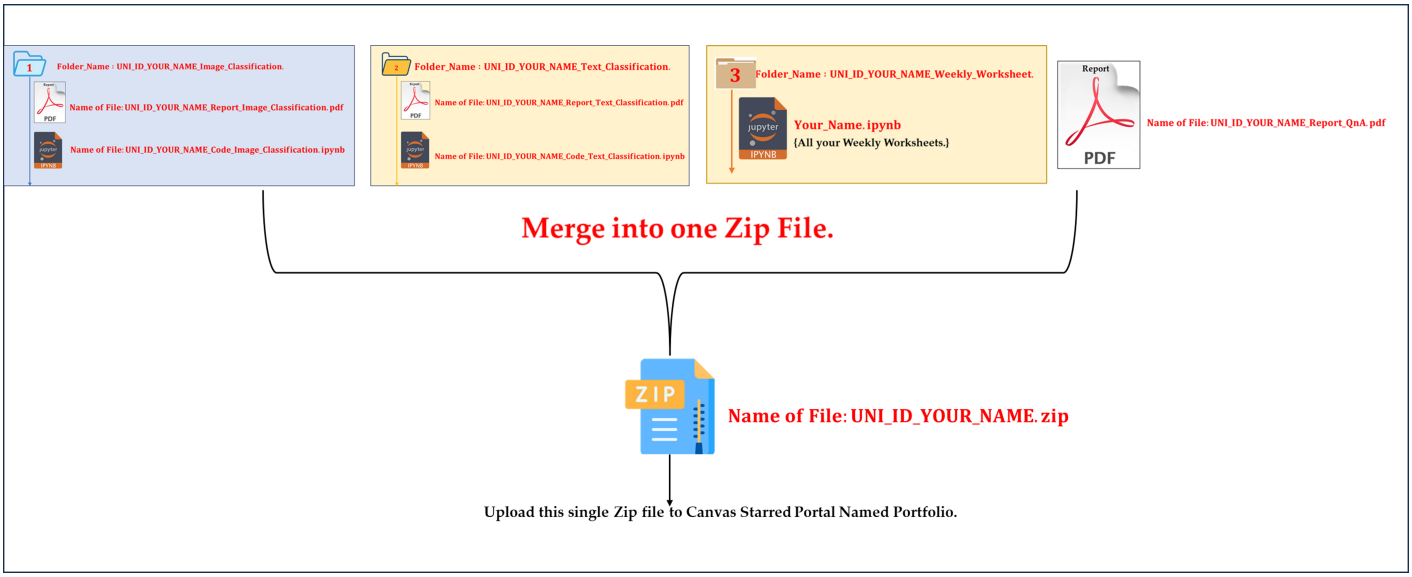

Figure 1: How to Make a Final Submission!!!